KY-006 / HW-508

Passive Piezo Buzzer Module Datasheet

Rev. 2026.1

The **KY-006** is a 3-pin passive piezo buzzer module designed for microcontrollers like Arduino, ESP32/ESP8266, and Raspberry Pi. Unlike active buzzers, it has no built-in oscillator. It requires an external square-wave signal (e.g., using the Arduino `tone()` function) to generate sound, allowing it to play beeps, alarms, and full melodies at various pitches.

Specifications

Parameter	Value / Description
Type	Passive piezoelectric buzzer (requires external square-wave)
Operating Voltage	1.5V – 15V (5V typical)
Resonant Frequency	≈ 2 kHz (loudest); plays a wide range of tones
Coil Resistance	≈ 42 Ω
Alternative Names	KY006, HW-508
Dimensions	18.5 mm × 15 mm

Pinout & Connection

The module features three 2.54 mm male header pins. Connect the signal pin directly to a digital output capable of generating a PWM or square wave.

Label	Function	Arduino Connection
S	Signal (Square wave input)	Any digital pin (e.g., Pin 8)
Middle	Not Connected (NC)	Leave unconnected
–	Ground	GND

> **How it Works:**
> A passive piezo buzzer produces sound when an alternating square wave switches HIGH/LOW at a specific frequency. On an Arduino, `tone(pin, frequency)` generates this wave automatically, and `noTone(pin)` stops it. Driving it near **2000 Hz** yields the maximum volume due to resonance.

Active vs. Passive

An active buzzer (like the KY-012) contains an internal oscillator—simply applying power makes it beep at

Arduino Implementation Examples

1. Basic Beep (Testing Wiring)

Plays two alternating tones: 1 kHz and 2 kHz (near resonance for maximum volume).

```
const int buzzer = 8;  // KY-006 S pin

void setup() {
  // nothing to configure — tone() handles the pin
}

void loop() {
  tone(buzzer, 1000);   // 1 kHz tone
  delay(500);
  tone(buzzer, 2000);   // 2 kHz tone (loudest)
  delay(500);
  noTone(buzzer);       // silence
  delay(1000);
}
```

2. Playing a Melody

Drives the buzzer through an array of notes and durations. Adding a short gap between notes prevents consecutive identical pitches from blurring together.

```
const int buzzer = 8;

#define NOTE_C4 262
#define NOTE_D4 294
#define NOTE_E4 330

// "Mary Had a Little Lamb" opening phrase
const int notes[]   = { NOTE_E4, NOTE_D4, NOTE_C4, NOTE_D4, NOTE_E4, NOTE_E4, NOTE_E4 };
const int lengths[] = {       4,       4,       4,       4,       4,       4,       2 };

void setup() {
  for (int i = 0; i < 7; i++) {
    int ms = 1000 / lengths[i];  // quarter note = 250 ms
    tone(buzzer, notes[i], ms);
    delay(ms + ms / 3);          // note duration + gap
    noTone(buzzer);
  }
}

void loop() {}
```

3. Siren / Frequency Sweep

Sweeps the frequency up and down—a classic siren effect demonstrating variable pitch control.

ESP32 Implementation (LEDC API)

Arduino's standard `tone()` function is not available on most ESP32 cores. Instead, use the hardware-based `ledc` API.

```
// ESP32: ledcWriteTone() replaces tone()
const int buzzer  = 8;   // KY-006 S pin
const int channel = 0;   // ledc channel (0-7)

void setup() {
  ledcAttachChannel(buzzer, 2000, 8, channel);
}

void loop() {
  ledcWriteTone(channel, 1000);  // 1 kHz tone
  delay(500);
  ledcWriteTone(channel, 2000);  // 2 kHz tone (near resonance)
  delay(500);
  ledcWriteTone(channel, 0);     // silence
  delay(1000);
}
```

Typical Applications

- **Alarms and sirens:** Rising/falling sweeps for security and warning systems.
- **Melodies and music:** Audio cues in games or musical toys.
- **UI audio feedback:** Distinctive beeps for button presses or error states.
- **Morse code:** Encoding messages via variable tone timings.

Comparison: KY-006 vs KY-012

Feature	KY-006 (Passive)	KY-012 (Active)
Requires `tone()` signal	Yes	No (Just DC power)
Fixed pitch only	No	Yes
Plays melodies	Yes	No
Best Used For	Music, alarms, pitch control	Single, simple fixed beep

Troubleshooting & FAQ

Why is my passive buzzer so quiet?

Drive it near its resonant frequency (~2 kHz) for maximum output. If more volume is needed, buffer the signal pin with a small NPN transistor (like 2N2222) to supply more current than a direct GPIO pin can provide.